

OpenWrt部署GRN client---刘世伟

前言

此文档是《OpenWrt部署GRN client》的姊妹篇，是按照《OpenWrt部署GRN client》步骤，在openwrt上部署grn client，中间遇到了一些问题，对某些操作进行修正，大部分雷同，差异部分为浅黄字体，请参考。

初始化系统

硬件设备

路由器：GL-MT6000

连接上网

1. 路由器电源上电并启动，找到路由器WAN口连接互联网网线。
2. 连接路由器WiFi，查看路由器底部标签，连接底部标签WiFi SSID（WiFi名称）和输入WiFi密码。
3. 设置管理员账户(root)和账户密码。ssh登录host为192.168.8.1，账户为root/密码。
4. 路由器控制面板地址：<https://192.168.8.1>，使用管理员账户登录。

升级系统

部署环境

路由器：GL-MT6000

操作系统：4.8.3-op24（OpenWrt 24.10.4）

CPU 架构：aarch64_cortex-a53

升级系统

GL-MT6000路由器的原装操作系统为OpenWrt 21.02-SNAPSHOT，仅支持到python3.9，而当前代码在python3.12环境下开发，存在兼容问题。需升级系统。

1. 下载4.8.3-op24固件

<https://dl.gl-inet.cn/router/mt6000/openwrt24>

2. 升级

进入路由器管理面板192.168.8.1，系统-升级-固件本地升级，上传下载的bin文件，安装升级

3. 若不成功，可尝试按照文档uboot

<https://docs.gl-inet.cn/router/4/features/uboot/>

部署本地前端

步骤

1. 打包前端

代码块

```
1  git clone git@github.com:FS-GRN-Dev/grn-frontend-openwrt.git
2  cd grn-frontend-openwrt
3  npm install
4  npm run build
```

2. 复制到openwrt

代码块

```
1  ssh root@192.168.8.1 "mkdir -p /www/grn_client" && scp -O -r dist
   root@192.168.8.1:/www/grn_client/
```

3. 更改文件权限

代码块

```
1  chmod -R 755 /www/grn_client
```

4. 创建 Nginx 配置

代码块

```
1  vim /etc/nginx/conf.d/grn_client.conf
```

参考 `GRN_Frontend/frontend-client/openwrt-nginx.conf`，修改`your_server_ip`为管理面地址（例如：36.134.205.22），端口为管理面后端端口（例如：8888）。浅黄部分配置nginx连接后端的连接池（不配置连接池，nginx和后端请求为短连接，出现大量TIME_WAIT），但是openwrt默认安装的nginx不支持keepalive指令配置，需要升级或者更换版本。

代码块

```
1  # 上游服务连接池
2  upstream backend_8080 {
3      server your_server_ip:8080;
4
5      keepalive 32; # 开启连接池，保持32个长连接（根据QPS调整，建议10-50）
```

```
6     keepalive_timeout 60s; # 连接池中空闲连接超时时间 (60秒无请求则关闭)
7     keepalive_requests 100; # 每个长连接最多处理100次请求, 避免单连接过载
8 }
9
10 # HTTPS 前端服务器
11 server {
12     listen 5177 ssl;
13     listen [::]:5177 ssl;
14     server_name localhost;
15
16     root /www/grn_client/dist;
17     index index.html;
18
19     # SSL 证书
20     ssl_certificate /etc/nginx/ssl/nginx.crt;
21     ssl_certificate_key /etc/nginx/ssl/nginx.key;
22
23     ssl_protocols TLSv1.2 TLSv1.3;
24     ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512:ECDHE-
RSA-AES256-GCM-SHA384:DHE-RSA-AES256-GCM-SHA384;
25     ssl_prefer_server_ciphers on;
26     ssl_session_cache shared:SSL:10m;
27     ssl_session_timeout 10m;
28
29     # 前端路由
30     location / {
31         try_files $uri $uri/ /index.html;
32     }
33
34     # 静态资源缓存
35     location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
36         expires 1y;
37         add_header Cache-Control "public, immutable";
38     }
39
40     # /info 代理
41     location = /info {
42         proxy_pass http://127.0.0.1:50080/info;
43         proxy_set_header Host $host;
44         proxy_set_header X-Real-IP $remote_addr;
45         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
46     }
47
48     # API 接口代理 (长连接 + 跨域)
49     location /api/ {
50         proxy_pass http://backend_8080$request_uri;
51         proxy_http_version 1.1;
```

```

52     proxy_set_header Host $host;
53     proxy_set_header X-Real-IP $remote_addr;
54     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
55     proxy_set_header X-Forwarded-Proto $scheme;
56
57     proxy_connect_timeout 30s;
58     proxy_send_timeout 30s;
59     proxy_read_timeout 60s;
60
61     add_header Access-Control-Allow-Origin *;
62     add_header Access-Control-Allow-Methods 'GET, POST, PUT, DELETE,
OPTIONS';
63     add_header Access-Control-Allow-Headers 'Content-Type, Authorization';
64
65     if ($request_method = 'OPTIONS') {
66         return 204;
67     }
68 }
69
70 # 安全头部
71 add_header X-Frame-Options "SAMEORIGIN" always;
72 add_header X-Content-Type-Options "nosniff" always;
73 add_header X-XSS-Protection "1; mode=block" always;
74 add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;
75 }

```

5. 生成 TLS 证书。自签名证书是不安全，生产版本必须配置私有CA证书或者公共CA证书。以下为生成私有CA证书和服务端证书的步骤（把生成的ca.crt导入浏览器）。

代码块

```

1  #申请私有CA证书
2  # 创建证书存储目录，按需修改路径
3  mkdir -p /etc/nginx/ssl
4  cd /etc/nginx/ssl
5
6  # 1. 生成CA私钥（EC算法，secp256r1是通用安全曲线）
7  openssl ecparam -genkey -name secp256r1 -out ca.key -noout
8
9  # 设置私钥权限（必须600，避免泄露）
10 chmod 600 ca.key
11
12 # 2. 生成CA根证书（自签名，有效期10年，适配内网长期使用）
13 openssl req -x509 -new -nodes -key ca.key -sha256 -days 3650 -out ca.crt -subj
"/C=CN/ST=Henan/L=Zhengzhou/O=GRN/OU=IT/CN=localhost"

```

```

14
15 # 3. 生成服务器私钥
16 openssl ecparam -genkey -name secp256r1 -out nginx.key -noout
17 chmod 600 nginx.key
18
19 #3.1 生成 PKCS8 私钥 (grn-backend的grpc服务器需要PKCS8格式私钥, secp256r1 =
    prime256v1)
20 #openssl ecparam -name prime256v1 -genkey -noout | openssl pkcs8 -topk8 -
    nocrypt -out server.key
21 #chmod 600 server.key
22
23 # 4. 生成证书签名请求 (CSR), 核心是指定内网域名 # 关键: subjectAltName补充域名/IP
    (多个用逗号分隔, 如你的服务器内网IP)
24 openssl req -new -key nginx.key -out nginx.csr -subj
    "/C=CN/ST=Henan/L=Zhengzhou/O=GRN/OU=IT/CN=localhost" -addext
    "subjectAltName=DNS:localhost,IP:192.168.8.1"
25
26 # 5. 用CA根证书签发服务器证书 (有效期10年, 可按需调整)
27 openssl x509 -req -in nginx.csr -CA /etc/nginx/ssl/ca.crt -CAkey
    /etc/nginx/ssl/ca.key -CAcreateserial -out nginx.crt -days 3650 -sha256 -
    extfile <(echo "subjectAltName=DNS:localhost,IP:192.168.8.1")
28
29 # 6. (可选) 将服务器证书 + 中间 CA 根证书拼接 (部分服务, 服务器需要加载完整链)
30 #cat nginx.crt intermediate_ca.crt > fullchain.crt
31
32 # 7. (可选) 查看证书完整信息 (包含所有字段, 可翻找 CN/SAN)
33 #openssl x509 -in nginx.crt -noout --text

```

备注：目前GRN数据面、管理面都使用同一套单向TLS证书，客户端校验服务器（所有服务器证书都是由上面私有CA证书进行签发），上面的CA证书只需生成一次，下面的证书按需生成多套。

openwrt本地前端证书（openwrt frontend nginx 证书），由上面步骤生成nginx.crt和nginx.key。

管理面前端证书（mangement frontend nginx 证书），由上面步骤生成nginx.crt和nginx.key。

管理面后端的GRPC证书（grpc server 证书），由上面步骤生成server.crt和server.key。

访问openwrt前端和管理面前端，把CA证书导入浏览器。

数据面和管理面开启GRPC通信，数据面加载CA证书，管理面加载server.crt和server.key。

6. 启用配置

代码块

```
1 nginx -t
```

```
2 /etc/init.d/nginx restart
```

7. 访问 <https://192.168.8.1:5177/>

8. 登录账户：admin/123456

部署GRN_Node

步骤

1. 安装 Docker

代码块

```
1  opkg update
2  opkg install docker dockerd docker-compose
```

2. 启动 Docker 服务

代码块

```
1  /etc/init.d/dockerd enable
2  /etc/init.d/dockerd start
```

3. 开发机上打包&拷贝

- a. 刘可盈发现使用默认NodeClient的tor Dockerfile构建的镜像层数太多，导入openwrt时，会出现 no space left on space，磁盘空间不足的问题，因此需要减少镜像层数（layer）。以下为最终的Dockerfile，构建镜像为三层，基础层，COPY层，RUN层，构建出来的镜像包grn-tor.tar为67.6M，导入后镜像大小为128M，占用磁盘为27%（7.2G）。

- b. 使用如下Dockerfile构建镜像

代码块

```
1  FROM debian:12.7-slim
2
3  ENV DEBIAN_FRONTEND=noninteractive
4
5  # 层 1: 文件拷贝（仅 1 层）
6  COPY entrypoint.sh cleanup_da.sh torrc.* /opt/
7
8  # 层 2: 安装 + 配置 + 权限 + 清理（仅 1 层）
9  RUN apt-get update && \
10     apt-get install -y --no-install-recommends \
11     ca-certificates \
```

```

12     apt-transport-https wget gpg \
13     iproute2 iputils-ping sudo vim procps \
14     bash-completion curl tcpdump telnet screen && \
15
16     # Tor 源配置
17     echo "deb [signed-by=/usr/share/keyrings/deb.torproject.org-
keyring.gpg] https://deb.torproject.org/torproject.org bookworm main" >>
/etc/apt/sources.list.d/tor.list && \
18     echo "deb-src [signed-by=/usr/share/keyrings/deb.torproject.org-
keyring.gpg] https://deb.torproject.org/torproject.org bookworm main" >>
/etc/apt/sources.list.d/tor.list && \
19     wget -qO-
https://deb.torproject.org/torproject.org/A3C4F0F979CAA22CDBA8F512EE8CBC9E88
6DDD89.asc \
20     | gpg --dearmor \
21     | tee /usr/share/keyrings/deb.torproject.org-keyring.gpg >/dev/null
&& \
22
23     # 安装 Tor
24     apt-get update && \
25     apt-get install -y --no-install-recommends tor deb.torproject.org-
keyring nix && \
26
27     # 目录权限
28     mkdir -p /var/lib/tor/.tor/keys && \
29     chown -R debian-tor:debian-tor /var/lib/tor && \
30     chmod 2700 /var/lib/tor/.tor /var/lib/tor/.tor/keys && \
31
32     # 脚本权限 + 格式修复
33     chmod +x /opt/entrypoint.sh /opt/cleanup_da.sh && \
34     sed -i 's/\r$//' /opt/entrypoint.sh /opt/cleanup_da.sh && \
35
36     # 终极清理（体积最小化）
37     apt-get clean && \
38     rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/*
39
40     # 启动入口
41     ENTRYPOINT ["/bin/bash", "/opt/entrypoint.sh"]

```

c. 构造打包镜像

代码块

```

1  cd GRN_Node/src/docker
2
3  # 1. 安装并配置 binfmt_misc, 让 Docker 支持多架构镜像的运行/构建

```

```
4  docker run --privileged --rm tonistiigi/binfmt --install all
5
6  # 2. 创建并切换到 buildx 构建器
7  docker buildx create --name mybuilder --use
8
9  # 3. 执行跨架构构建镜像
10 docker buildx build \
11 --platform linux/arm64/v8 \
12 -t grn-tor \
13 --load .
14
15 # 4. 导出镜像
16 docker save -o grn-tor.tar grn-tor:latest
```

d. 压缩打包并拷贝到路由器

代码块

```
1  tar -czvf GRN_Node.tar.gz GRN_Node
2  scp xx/GRN_Node.tar.gz root@192.168.8.1:/root
```

4. 解压GRN_Node.tar.gz

代码块

```
1  tar -xzvf GRN_Node.tar.gz
```

5. 安装python3

代码块

```
1  opkg install python3 python3-pip python3-logging
```

6. 进python虚拟环境

代码块

```
1  pip3 install --no-cache-dir virtualenv
2  virtualenv venv
3  source venv/bin/activate
```

7. 安装依赖

- a. grpcio、grpcio-tools、protobuf、psutil都依赖 C++ 扩展，编译会失败，需使用预编译 wheel 包

代码块

```
1  pip install wheel
2  pip install grpcio==1.78.0 \
3      -i https://mirrors.aliyun.com/pypi/simple/ \
4      --extra-index-url https://www.piwheels.org/simple \
5      --only-binary=:all: \
6      --no-build-isolation
7  pip install grpcio-tools==1.78.0 \
8      -i https://mirrors.aliyun.com/pypi/simple/ \
9      --extra-index-url https://www.piwheels.org/simple \
10     --only-binary=:all: \
11     --no-build-isolation
12 pip install psutil==7.2.0 \
13     -i https://mirrors.aliyun.com/pypi/simple/ \
14     --extra-index-url https://www.piwheels.org/simple \
15     --only-binary=:all: \
16     --no-build-isolation
```

- b. 将GRN_Node下的requirements.txt中的pycurl注释掉，修改psutil的版本为7.2.0，安装其余的依赖包

代码块

```
1  # GRPC 核心（升级到1.78.0）
2  grpcio==1.78.0
3  grpcio-tools==1.78.0
4
5  # Protobuf（适配grpcio-tools 1.78.0）
6  protobuf==6.31.1
7
8  # 基础工具（升级到grpcio要求的版本）
9  setuptools>=77.0.1
10
11 # 其他依赖（保持原有版本，无冲突）
12 stem==1.8.2
13 pycurl==7.45.4
14 psutil==7.2.0
15 docker>=6.1.3
16 PySocks>=1.7.1
17
18 # mypy-protobuf（适配protobuf 6.x）
19 mypy-protobuf==3.6.0
```

```
20
21 # 测试依赖（保持兼容）
22 pytest>=7.4.0
23 pytest-asyncio>=0.21.0
24 pytest-dependency>=0.5.1
```

代码块

```
1 pip install --no-cache-dir -r requirements.txt
```

1. 交叉编译pycurl

通过pip安装pycurl需要依赖curl-dev，而OpenWrt 默认包源没有 curl-dev，不可行。通过opkg 安装的python3-curl，存在ssl后端不匹配的问题，不可行。因此需要交叉编译pycurl，并指定ssl 后端为openssl。

a. 编译（此步骤只需编译第一次，后续直接安装编译好的python3-curl_7.45.2-r3_aarch64_cortex-a53.ipk）

i. Ubuntu 22.04开发机上，安装依赖

代码块

```
1 sudo apt update
2 sudo apt install -y \
3 build-essential clang flex bison g++ gawk gcc-multilib g++-multilib \
4 gettext git libncurses-dev libssl-dev python3 python3-distutils \
5 rsync unzip zlib1g-dev file wget libelf-dev ecj fastjar java-propose-
  classpath \
6 libpam0g-dev ccache curl python3-setuptools swig
```

ii. 下载OpenWrt SDK

代码块

```
1 wget
  https://downloads.openwrt.org/releases/24.10.4/targets/mediatek/filogic/o
  penwrt-sdk-24.10.4-mediatek-filogic_gcc-13.3.0_musl.Linux-x86_64.tar.zst
2 tar --zstd -xf openwrt-sdk-24.10.4-mediatek-filogic_gcc-
  13.3.0_musl.Linux-x86_64.tar.zst
3 cd openwrt-sdk-24.10.4-mediatek-filogic_gcc-13.3.0_musl.Linux-x86_64
```

iii. 初始化

```
1 1. run ./scripts/feeds update -a
2  ./scripts/feeds install -a
```

iv. 配置

代码块

```
1  make menuconfig
2  #勾选pycurl
3  Languages --->
4      Python --->
5          <*> python3
6          <M> python3-curl
7  #选择OpenSSL
8  Libraries --->
9      libcurl --->
10         Selected SSL library (OpenSSL) --->
11             ( ) mbed TLS
12             ( ) wolfSSL
13             (X) OpenSSL
14             ( ) GNUTLS
15             ( ) No SSL support
```

v. 刷新配置

代码块

```
1  make defconfig
```

vi. 安装python-curl

代码块

```
1  ./scripts/feeds install python-curl
```

vii. 指定SSL library

代码块

```
1  export PYCURL_SSL_LIBRARY=openssl
```

viii. 编译python-curl

代码块

```
1 make package/feeds/packages/python-curl/compile -j$(nproc) V=s
```

ix. 生成的 ipk 文件 `/bin/packages/aarch64_cortex-a53/packages/python3-curl_7.45.2-r3_aarch64_cortex-a53.ipk`，拷贝到openwrt

b. openwrt安装python3-curl

代码块

```
1 opkg install ./python3-curl_7.45.2-r3_aarch64_cortex-a53.ipk
```

c. 添加系统模块路径到虚拟环境

代码块

```
1 cd venv/lib/python3.11/site-packages/  
2 echo "/usr/lib/python3.11/site-packages/" > sys_packages.pth
```

2. 导入镜像

代码块

```
1 cd src/docker  
2 docker load -i grn-tor.tar
```

新的镜像的导入

代码块

```
1 root@GL-MT6000:/overlay# docker load -i grn-tor.tar  
2 f0f039847c08: Loading layer [=====>] 29.16MB/29.16MB  
3 76790b4c179a: Loading layer [=====>] 2.782kB/2.782kB  
4 12fcf9885076: Loading layer [=====>] 41.72MB/41.72MB  
5 Loaded image: grn-tor:latest
```

3. 修改docker网络模式 (bridge-->host)

代码块

```
1 #GRN_Node/src/config/config.py
2 class ContainerConfig:
3     DEFAULT_IMAGE = "grn-tor"
4     DEFAULT_NETWORK_MODE = "host"
```

4. 运行node_client

代码块

```
1 source venv/bin/activate
2 cd GRN_Node/src/core
3 python node_client.py -s <服务端地址> -r openwrt
```

脱离终端，后台运行

代码块

```
1 (python /root/GRN_Node/src/core/node_client.py -s <服务端地址> -r openwrt 2>&1
  &)
```

透明代理

sing-box

1. 安装sing-box

代码块

```
1 opkg install sing-box
```

2. openwrt端sing-box默认配置如下：（管理面前端点击多链路启用时，自动生成运行配置）

代码块

```
1 {
2     "log": {
3         "level": "info",
4         "timestamp": true
5     },
6     "experimental": {
7         "clash_api": {
8             "external_controller": "127.0.0.1:9090",
9             "secret": "grn-secret"
10        }
11    }
```

```
11 },
12 "inbounds": [
13   {
14     "type": "redirect",
15     "tag": "redirect-in",
16     "listen": "0.0.0.0",
17     "listen_port": 11090
18   }
19 ],
20 "outbounds": [
21   {
22     "type": "selector",
23     "tag": "out-select",
24     "outbounds": ["direct", "mptcp-tunnel", "socks5"],
25     "default": "direct",
26     "interrupt_exist_connections": true
27   },
28   {
29     "type": "direct",
30     "tag": "direct"
31   },
32   {
33     "type": "urltest",
34     "tag": "mptcp-tunnel",
35     "outbounds": ["mptcp-path-1", "mptcp-path-2", "mptcp-path-3"],
36     "url": "http://www.gstatic.com/generate_204",
37     "interval": "1m",
38     "tolerance": 50,
39     "idle_timeout": "168h",
40     "interrupt_exist_connections": true
41   },
42   {
43     "type": "vless",
44     "tag": "mptcp-path-1",
45     "server": "10.0.1.1",
46     "server_port": 11091,
47     "uuid": "UUID",
48     "network": "tcp",
49     "tcp_multi_path": true,
50     "connect_timeout": "5s",
51     "multiplex": {
52       "enabled": true,
53       "protocol": "h2mux",
54       "max_connections": 4,
55       "min_streams": 4,
56       "padding": false
57     }
58   }
59 ]
```

```
58     },
59     {
60         "type": "vless",
61         "tag": "mptcp-path-2",
62         "server": "10.0.2.1",
63         "server_port": 11091,
64         "uuid": "UUID",
65         "network": "tcp",
66         "tcp_multi_path": true,
67         "connect_timeout": "5s",
68         "multiplex": {
69             "enabled": true,
70             "protocol": "h2mux",
71             "max_connections": 4,
72             "min_streams": 4,
73             "padding": false
74         }
75     },
76     {
77         "type": "vless",
78         "tag": "mptcp-path-3",
79         "server": "10.0.3.1",
80         "server_port": 11091,
81         "uuid": "UUID",
82         "network": "tcp",
83         "tcp_multi_path": true,
84         "connect_timeout": "5s",
85         "multiplex": {
86             "enabled": true,
87             "protocol": "h2mux",
88             "max_connections": 4,
89             "min_streams": 4,
90             "padding": false
91         }
92     },
93     {
94         "type": "socks",
95         "tag": "socks5",
96         "server": "127.0.0.1",
97         "server_port": 9050,
98         "version": "5",
99         "network": "tcp"
100    }
101 ],
102 "route": {
103     "final": "out-select",
104     "auto_detect_interface": true
```

```
105     }
106 }
107
```

3. openwrt端sing-box运行 (sing-box由node_client.py自动启动与停止, 与GRN_Node生命周期一致)

4. iptable劫持流量

a. openwrt流量劫持脚本 grn-iptables.sh (此脚本直接由GRN_Node自动调用, 无需手动操作)

代码块

```
1  #!/bin/sh
2
3  CHAIN="GRN"
4  LAN_IF="br-lan"
5  TCP_PORT="11090"
6  #DNS_PORT="50053"
7
8  add_rules() {
9      echo "[+] Adding iptables rules..."
10
11     # 创建链 (如果不存在)
12     iptables -t nat -N $CHAIN 2>/dev/null
13
14     # 清空链
15     iptables -t nat -F $CHAIN
16
17     # 排除内网
18     iptables -t nat -A $CHAIN -d 127.0.0.0/8 -j RETURN
19     iptables -t nat -A $CHAIN -d 192.168.0.0/16 -j RETURN
20     iptables -t nat -A $CHAIN -d 10.0.0.0/8 -j RETURN
21     iptables -t nat -A $CHAIN -d 172.16.0.0/12 -j RETURN
22
23     # 排除 bogon / 保留地址段 (DNS 劫持等会导致请求发往这些地址)
24     iptables -t nat -A $CHAIN -d 0.0.0.0/8 -j RETURN
25     iptables -t nat -A $CHAIN -d 169.254.0.0/16 -j RETURN
26     iptables -t nat -A $CHAIN -d 224.0.0.0/4 -j RETURN
27     iptables -t nat -A $CHAIN -d 240.0.0.0/4 -j RETURN
28
29     # TCP 转发
30     iptables -t nat -A $CHAIN -p tcp -j REDIRECT --to-ports $TCP_PORT
31
32     # 挂 PREROUTING (避免重复添加)
33     iptables -t nat -C PREROUTING -i $LAN_IF -p tcp -j $CHAIN 2>/dev/null
|| \
```



```
34     iptables -t nat -A PREROUTING -i $LAN_IF -p tcp -j $CHAIN
35
36     # DNS 劫持 (避免重复)
37     #iptables -t nat -C PREROUTING -i $LAN_IF -p udp --dport 53 -j REDIRECT
--to-ports $DNS_PORT 2>/dev/null || \
38     #iptables -t nat -A PREROUTING -i $LAN_IF -p udp --dport 53 -j REDIRECT
--to-ports $DNS_PORT
39
40     echo "[+] Done."
41 }
42
43 del_rules() {
44     echo "[-] Deleting iptables rules..."
45
46     # 删除 PREROUTING 规则
47     while iptables -t nat -C PREROUTING -i $LAN_IF -p tcp -j $CHAIN
2>/dev/null; do
48         iptables -t nat -D PREROUTING -i $LAN_IF -p tcp -j $CHAIN
49     done
50
51     #while iptables -t nat -C PREROUTING -i $LAN_IF -p udp --dport 53 -j
REDIRECT --to-ports $DNS_PORT 2>/dev/null; do
52     #     iptables -t nat -D PREROUTING -i $LAN_IF -p udp --dport 53 -j
REDIRECT --to-ports $DNS_PORT
53     #done
54
55     # 清空并删除链
56     iptables -t nat -F $CHAIN 2>/dev/null
57     iptables -t nat -X $CHAIN 2>/dev/null
58
59     echo "[-] Done."
60 }
61
62 status_rules() {
63     echo "[*] Current NAT rules:"
64     iptables -t nat -L -n -v --line-numbers
65 }
66
67 case "$1" in
68     start)
69         add_rules
70         ;;
71     stop)
72         del_rules
73         ;;
74     restart)
75         del_rules
```

```

76         add_rules
77         ;;
78     status)
79         status_rules
80         ;;
81     *)
82         echo "Usage: $0 {start|stop|restart|status}"
83         exit 1
84         ;;
85 esac

```

代码块

```

1  chmod +x /root/GRN_Node/scripts/grn-iptables.sh #执行权限
2  grn-iptables.sh start #启动
3  grn-iptables.sh stop #停止
4  grn-iptables.sh status #查看iptables状态

```

Supervisor监控node_client.py运行

Supervisor

OpenWrt 没有官方 `supervisor` 包，一般用已有的 `venv + pip` 安装，再用 `procd init` 让 `supervisord` 开机自启（VPS上根本不用`procd init`开机自启 `supervisord`，VPS上有`systemd`管理 `supervisord`）；日常用 `supervisorctl` 管 `node_client.py`。

1. 安装Supervisor到venv下

代码块

```

1  source /root/venv/bin/activate
2  pip install supervisor
3
4  # 确认命令在 venv 里
5  (venv) root@GL-MT6000:~# which supervisord
6  /root/venv/bin/supervisord
7
8  (venv) root@GL-MT6000:~# which supervisorctl
9  /root/venv/bin/supervisorctl

```

2. 生成supervisor配置目录与主配置

代码块

```

1  mkdir -p /etc/supervisor/conf.d /var/log/supervisor

```

```
2
3 # 使用 supervisor 小工具 echo_supervisord_conf 生成默认主配置
4 /root/venv/bin/echo_supervisord_conf > /etc/supervisor/supervisord.conf
```

编辑 `/etc/supervisor/supervisord.conf`（开头分号是注释），至少改这几处：

代码块

```
1 [unix_http_server]
2 file=/var/run/supervisor.sock
3 chmod=0700
4
5 [supervisord]
6 logfile=/var/log/supervisor/supervisord.log
7 pidfile=/var/run/supervisord.pid
8 childlogdir=/var/log/supervisor
9 nodaemon=false
10
11 [rpcinterface:supervisor]
12 supervisor.rpcinterface_factory =
    supervisor.rpcinterface:make_main_rpcinterface
13
14 [supervisorctl]
15 serverurl=unix:///var/run/supervisor.sock
16
17 ; 关键这行，加载 conf.d 下的程序配置
18 [include]
19 files = /etc/supervisor/conf.d/*.conf
```

可不启用 Web UI（`[inet_http_server]`），省内存、也更安全。

3. 配置 `node_client` 程序

代码块

```
1 cat > /etc/supervisor/conf.d/node_client.conf << 'EOF'
2 [program:node_client]
3 command=/root/venv/bin/python /root/GRN_Node/src/core/node_client.py -s
    36.134.205.22:50050 -r openwrt
4 directory=/root/GRN_Node
5 autostart=true
6 autorestart=true
7 startsecs=5
8 startretries=10
9 stopwaitsecs=20
10 stopsignal=TERM
```

```

11 stopasgroup=true
12 killasgroup=true
13 redirect_stderr=true
14 stdout_logfile=/var/log/supervisor/node_client.log
15 stdout_logfile_maxbytes=5MB
16 stdout_logfile_backups=5
17 environment=VIRTUAL_ENV="/root/venv",PATH="/root/venv/bin:%
  (ENV_PATH)s",PYTHONPATH="/root/GRN_Node/src"
18 EOF

```

要点：

1> `venv python` 链到系统 Python，必须带 `VIRTUAL_ENV` / `PATH`，否则依赖可能找不到。

2> `autorestart=true`：崩溃会自动拉起。

3> 如果要改服务器地址（36.134.205.22:50050），改完这个conf后，执行：

`/root/venv/bin/supervisorctl update` 即可。

4. 用 OpenWrt procd init 拉起 supervisord（开机自启）

代码块

```

1 cat > /etc/init.d/supervisord << 'EOF'
2 #!/bin/sh /etc/rc.common
3
4 START=99
5 STOP=10
6 USE_PROCD=1
7
8 BIN="/root/venv/bin/supervisord"
9 CONF="/etc/supervisor/supervisord.conf"
10
11 start_service() {
12     mkdir -p /var/log/supervisor /var/run
13
14     procd_open_instance
15     procd_set_param command "$BIN" -n -c "$CONF"
16     procd_set_param respawn 3600 5 5
17     procd_set_param stdout 1
18     procd_set_param stderr 1
19     procd_close_instance
20 }
21 EOF

```

supervisord添加权限并开机自启

代码块

```
1  chmod +x /etc/init.d/supervisord
2  /etc/init.d/supervisord enable
3  /etc/init.d/supervisord start
```

supervisord停止开机自启（当然也会停掉node_client.py）

代码块

```
1  /etc/init.d/supervisord stop
```

注意：不要用 `kill -9 supervisord` 测 procd 拉起崩溃的supervisord（如果要测，用kill `$(pidof supervisord)`），否则会留下占端口(例如：http 端口50080)的node_client.py 孤儿进程，会循环拉起node_client.py失败。此时，只能用/root/venv/bin/supervisorctl stop node_client 先停掉监控，再开启监控，/root/venv/bin/supervisorctl start node_client。

5. 日常管理（Supervisor 本体）

代码块

```
1  # 看状态
2  /root/venv/bin/supervisorctl status
3
4  # 启停重启
5  /root/venv/bin/supervisorctl start node_client
6  /root/venv/bin/supervisorctl stop node_client
7  /root/venv/bin/supervisorctl restart node_client
8
9  # 改完 conf 后重载
10 /root/venv/bin/supervisorctl reread
11 /root/venv/bin/supervisorctl update
12
13 # 看日志
14 tail -f /var/log/supervisor/node_client.log
```

可加别名（可选）：

代码块

```
1  echo 'alias sc="/root/venv/bin/supervisorctl"' >> /etc/profile
```

6. 验证openwrt重启是否生效

代码块

```
1  reboot
2
3  # 重启后:
4  ps | grep -E 'supervisord|node_client'
5
6  /root/venv/bin/supervisorctl status
7  # 期望类似:
8  node_client                                RUNNING    pid 16616, uptime 0:26:36
```

或者

代码块

```
1  kill $(pgrep -f 'node_client.py.*-s ') 2>/dev/null
2
3  /root/venv/bin/supervisorctl status
4  # 期望类似:
5  node_client                                STARTING
6
7  /root/venv/bin/supervisorctl status
8  # 期望类似:
9  node_client                                RUNNING    pid 15801, uptime 0:00:06
```

7. 主动停止node_client.py (停止监控)

代码块

```
1  /root/venv/bin/supervisorctl stop node_client
2  node_client: stopped
3
4
5  /root/venv/bin/supervisorctl status
6  # 期望类似:
7  node_client                                STOPPED    Aug 11 03:13 PM
8
9
10 # 重新监控
11 /root/venv/bin/supervisorctl start node_client
12 node_client: started
13
14
15 /root/venv/bin/supervisorctl status
```

```
16 # 期望类似:
17 node_client
```

```
RUNNING pid 21368, uptime 0:00:15
```

8. 问题排查，例如：若 `supervisorctl status` 报连不上 socket

代码块

```
1 ls -l /var/run/supervisor.sock
2
3 ps | grep supervisord
4
5 logread | tail -50
```

注意

1. openwrt默认安装了tor（在/etc/init.d/tor），torrc在/etc/tor/torrc，且是默认开机启动的，但是uci固件设置了

uci get tor.global.enable = 0，所以开启启动时，tor启动脚本退出了。要防止默认安装的tor和我们容器的tor 冲突。

2. 严禁按路由器reset键（重置键）10秒以上（按一下reset通常不会重启网络啥的），或者在控制面板（<http://192.168.8.1/>）上，系统-->重置固件，否则走的就是撤收流程。这会恢复出厂设置，所有安装的环境，软件，包，都会清除。如果要重启，可以ssh登录192.168.8.1，手动输入reboot 或者 断电重启。

问题

1. 透明代理只能处理TCP和DNS，不能处理非DNS的UDP
2. 目前所有测试节点均在国内，暂时未测试国外网站访问
3. 运行后，磁盘空间不足

已解决，缩减镜像layer数，避免空间膨胀

存储扩展

原配8GB闪存，若不扩展，运行时可能出现磁盘空间不足的问题。需将docker迁移到u盘

```

--- Logging error ---
Traceback (most recent call last):
  File "/usr/lib/python3.11/logging/handlers.py", line 73, in emit
    File "/usr/lib/python3.11/logging/handlers.py", line 197, in shouldRollover
OSError: [Errno 28] No space left on device
Call stack:
  File "/root/GRN_Node/src/core/node_client.py", line 301, in <module>
    main()
  File "/root/GRN_Node/src/core/node_client.py", line 294, in main
    asyncio.run(client.run())
  File "/usr/lib/python3.11/asyncio/runners.py", line 190, in run
  File "/usr/lib/python3.11/asyncio/runners.py", line 118, in run
  File "/usr/lib/python3.11/asyncio/base_events.py", line 641, in run_until_complete
  File "/usr/lib/python3.11/asyncio/base_events.py", line 608, in run_forever
  File "/usr/lib/python3.11/asyncio/base_events.py", line 1936, in _run_once
  File "/usr/lib/python3.11/asyncio/events.py", line 84, in _run
  File "/root/GRN_Node/src/core/grpc_handler.py", line 271, in _process_command
    response = await self.command_dispatcher.dispatch(
  File "/root/GRN_Node/src/core/command_dispatcher.py", line 111, in dispatch
    return await command_handler.execute(msg_data, command_id)
  File "/root/GRN_Node/src/commands/heart_beat_response_command.py", line 26, in execute
    self.logger.info("收到心跳响应")
Message: '收到心跳响应'
Arguments: ()

```

```

root@GL-MT6000: # df -h
Filesystem      Size      Used Available Use% Mounted on
/dev/root        56.5M     56.5M          0 100% /rom
tmpfs            491.5M    91.3M    400.2M   19% /tmp
/dev/loop0       7.2G      7.2G          0 100% /overlay
overlayfs:/overlay 7.2G      7.2G          0 100% /
tmpfs            512.0K      4.0K    508.0K    1% /dev
overlayfs:/overlay 7.2G      7.2G          0 100% /opt/docker

```

参考官方技术文档 https://docs.gl-inet.cn/router/4/features/memory_mount/。此文档将根文件系统迁移至外存储器，系统容易崩溃；因此只迁移docker数据

此型号无TF卡槽，仅有USB口，可插U盘/硬盘

1. 查看路由器是否识别U盘，查看挂载信息

代码块

```
1 df -h
```

2. 卸载“网络存储”功能

代码块

```
1 umount /dev/sda1
```

3. 安装相关工具

代码块

```
1 opkg update
2 opkg install block-mount kmod-usb-storage kmod-fs-ext4 e2fsprogs kmod-fs-vfat
```

4. 查看u盘的分区信息

代码块


```
1 blkid /dev/sda1
```

5. 格式化

代码块

```
1 mkfs.ext4 /dev/sda1 << EOF
2 EOF
```

6. 挂载 U 盘

代码块

```
1 mkdir -p /mnt/sda1
2 mount -t ext4 /dev/sda1 /mnt/sda1
```

7. 配置自动挂载

代码块

```
1 block detect > /etc/config/fstab
2 #查看当前配置
3 uci show fstab
4 #修改对应条目
5 uci set fstab.@mount[2].enabled='1'
6 uci set fstab.@mount[2].uuid='uuid-' #改成 blkid /dev/sda1 查到的uuid
7 uci commit fstab
```

8. 重启路由器

代码块

```
1 reboot
```

9. 验证扩展是否成功

代码块

```
1 df -h
```

10. 迁移docker

a. 移动数据

代码块

```
1 /etc/init.d/dockerd stop
2 mv /opt/docker /mnt/sda1/docker
```

b. 修改配置

代码块

```
1 vi /etc/config/dockerd
2 #改成以下配置
3 config globals 'globals'
4 # option alt_config_file '/etc/docker/daemon.json'
5 option data_root '/mnt/sda1/docker'
```

c. 重启docker

代码块

```
1 /etc/init.d/dockerd start
```

d. 验证

代码块

```
1 docker info | grep -E "Root|Storage"
2 #正确结果
3 Storage Driver: overlay2
4 Docker Root Dir: /mnt/sda1/docker
```